

Buffer allocation — the defensible core

What to be able to say, and why each part is true

RailCast · 8 September 2026

What this document is for. The three-part argument in section 3 is the whole proof that this is a formulation rather than a library call. It is what an interviewer will probe, and it is the part that must be reproducible on a whiteboard without notes. Everything else here is context for it.

1. The claim, in one paragraph

A timetable contains padding: scheduled running times exceed the minimum a train can actually achieve. That padding absorbs delay. It is currently distributed however the timetable happens to distribute it. *Given the same total padding*, is there a better allocation — one that reduces expected lateness?

This is posed as a linear program: the padding on each segment is a decision variable, the total is a budget constraint, delay propagation along the route is a set of linear constraints, and expected lateness across historical days is the objective. It is solved with HiGHS via `scipy.optimize.linprog`. The formulation is derived below; the solver is only the arithmetic.

2. The program

Stops $i = 0, 1, \dots, n$ along one train's route; segment i runs from stop $i - 1$ to stop i . Scenarios $\omega \in \Omega$ are historical days.

Data

$$\begin{aligned} m_i &= \text{minimum technical running time on segment } i \\ r_i &= \text{scheduled running time on segment } i \\ b_i^0 &= r_i - m_i \\ B &= \sum_{i=1}^n b_i^0 \\ \delta_i^\omega &= \max(0, \rho_i^\omega - m_i) \end{aligned}$$

ρ_i^ω is the observed running time on segment i on day ω ; m_i is estimated as the 5th percentile of observed running times; b_i^0 is the timetable's own allocation and is the baseline; B is the budget, so the same total is redistributed and journey time is unchanged; δ_i^ω is the primary delay actually incurred.

Variables, objective, constraints

$$b_i \geq 0 \quad (i = 1 \dots n), \quad L_i^\omega \geq 0 \quad (i = 1 \dots n, \omega \in \Omega)$$

$$\min_{b, L} \quad \frac{1}{|\Omega|} \sum_{\omega \in \Omega} \sum_{i=1}^n w_i L_i^\omega$$

$$\begin{aligned}
(\text{C1}) \quad L_i^\omega &\geq L_{i-1}^\omega + \delta_i^\omega - b_i & \forall i, \forall \omega \\
(\text{C2}) \quad L_i^\omega &\geq 0 & \forall i, \forall \omega \\
(\text{C3}) \quad \sum_{i=1}^n b_i &\leq B \\
(\text{C4}) \quad 0 \leq b_i &\leq \bar{b}_i & \forall i \\
(\text{C5}) \quad L_0^\omega &= \ell_0^\omega
\end{aligned}$$

3. Why the maximum can be replaced by two inequalities

This is the argument to know cold.

The problem

Delay propagation along a route is genuinely nonlinear:

$$L_i^\omega = \max(0, L_{i-1}^\omega + \delta_i^\omega - b_i).$$

A train carries its delay forward; buffer eats into it; and it cannot become “negatively late” in a way that lets it depart a scheduled stop early. That max cannot appear in a linear program.

Constraints (C1) and (C2) are the same expression written as two *inequalities*. They say L_i^ω is at least each of the two arguments — which is weaker than saying it equals their maximum. Every feasible point of the true system is feasible here, but so are points where L_i^ω is too large. The relaxation must be shown to be **exact**: that its optimal buffers and optimal cost are those of the true recursion.

The argument, in three steps

- 1. The objective pushes every L_i^ω down.** Each enters with coefficient $w_i/|\Omega| \geq 0$. Nothing in the program rewards a larger L_i^ω .
- 2. Its only lower bounds are (C1) and (C2).** No other constraint mentions L_i^ω from below. Driven down as far as feasibility allows, it comes to rest exactly at $\max(0, L_{i-1}^\omega + \delta_i^\omega - b_i)$ — the value the true recursion specifies.
- 3. Reducing it never costs anything elsewhere.** L_i^ω appears once more, on the right-hand side of (C1) for index $i+1$, with coefficient $+1$. Reducing L_i^ω *lowers* that bound, which *relaxes* the next constraint. Both forces act in the same direction. There is no configuration in which inflating L_i^ω buys a saving.

The formal version

Lemma (least element). Fix any buffer vector b and scenario ω , and let $\bar{L}^\omega(b)$ be the true recursion. Every L^ω satisfying (C1) and (C2) has $L_i^\omega \geq \bar{L}_i^\omega(b)$ at every stop.

Proof. By induction along the route. At $i=1$, (C1) and (C2) give $L_1^\omega \geq \max(0, L_0^\omega + \delta_1^\omega - b_1) = \bar{L}_1^\omega(b)$. If $L_{i-1}^\omega \geq \bar{L}_{i-1}^\omega(b)$ then $L_i^\omega \geq \max(0, L_{i-1}^\omega + \delta_i^\omega - b_i) \geq \max(0, \bar{L}_{i-1}^\omega(b) + \delta_i^\omega - b_i) = \bar{L}_i^\omega(b)$. $\square$

Theorem. If $w_i \geq 0$ for every i , the relaxed program and the true problem have the same optimal cost, and every optimal b^* of the relaxed program is optimal for the true problem.

Proof. For fixed b , the lemma and $w \geq 0$ give $\sum_i w_i L_i^\omega \geq \sum_i w_i \bar{L}_i^\omega(b)$ for every feasible L , and $\bar{L}(b)$ is itself feasible, so the minimum over L is exactly the true cost of b . Minimising over b as well gives both programs the same optimum, and $\text{true}(b^*) \leq \text{relaxed}(b^*, L^*) = \text{optimum} \leq \text{true}(b^*)$. $\square$

No positivity condition appears, and terminus-only weighting needs nothing extra.

What is not exact, and an earlier mistake

The theorem is about the buffers and the cost. It says nothing about the lateness values the solver returns, and they need not be the true ones. Replacing L^* by $\bar{L}(b^*)$ changes the objective by $\sum_i w_i (L_i^* - \bar{L}_i(b^*)) \geq 0$, which is zero at an optimum, so L_i^* is exact wherever $w_i > 0$ and unconstrained from above wherever $w_i = 0$.

An earlier version of this document claimed the lateness was exact at every stop i with $\sum_{j \geq i} w_j > 0$, because a reduction at i propagates forward to the terminus with coefficient 1. It does not: the propagation stops wherever the zero floor binds downstream, since an early train absorbs the excess. Three stops, terminus-only weights, $l_0 = 0$, $\delta = (10, 0, 5)$, budget 100 and $b = (0, 50, 50)$. The true lateness is $(10, 0, 0)$, yet $(30, 0, 0)$ is feasible and costs the same 0, which is the optimum. The condition held at stop 1 and the claim failed there. Nothing reported by the project depended on it, because every cost is recomputed from the buffers through the true recursion.

The version to give if pushed. For fixed buffers, the real lateness is the least point the constraints allow, so with non-negative weights the program can never look cheaper than reality. The buffers it returns are exactly optimal under either weighting. Lateness is never read off the solver.

4. The failure mode

Introduce a negative weight anywhere and the exactness breaks.

Suppose $w_k < 0$ at some stop, intended to reward early arrival. Then the objective *rewards* a larger L_k^ω . The solver will inflate it — feasibly, since (C1) and (C2) are only lower bounds — to collect that reward, and the constraint at $k + 1$ absorbs the inflation by raising L_{k+1}^ω , which may be cheap if the weights beyond k are small.

The lateness variables then no longer represent lateness. The reported objective is not the expected lateness of any timetable, and the buffer vector is optimal for a problem nobody posed.

The fix is not a negative weight. Rewarding earliness requires representing it as a separate non-negative variable — $E_i^\omega \geq (\text{scheduled}) - (\text{actual})$, carrying its own non-negative cost — so that both quantities are minimised and neither is inflated for gain.

5. Three methodological decisions

1. Significance is a paired bootstrap over *days*, not over rows

The plan says to evaluate against b^0 on held-out days. It does not say how to decide the difference is real, and with a handful of days an improvement of a few seconds is easily noise. The project already has the apparatus: the paired bootstrap veto used for model promotion.

One refinement matters. That test resamples *rows*. Here the independent unit is the **day**, not the stop-arrival: within one day, delay at consecutive stops is strongly correlated by construction — that is the propagation the whole model describes. Resampling stop-arrivals would treat correlated

observations as independent and understate the variance, producing a confident answer that is not warranted.

The number that governs the statistics is the number of days, not the number of variables. A 29-stop service with 32 complete days gives 896 lateness variables and **32** independent scenarios. Power comes from 32. Resample whole days, with replacement, and report the interval on the difference $(\text{cost}(b^*) - \text{cost}(b^0))$.

Consequence to accept in advance: 32 scenarios is a small sample for a significance claim, and the honest outcome may be “an improvement was measured and cannot be distinguished from noise.” The way to raise power is more days, and the data source supports it — the API serves history back to 2007, so the scenario set can be extended by backfilling more dates rather than by weakening the test.

2. The cap $\bar{b}_i$ needs defining, and reporting

$\bar{b}_i$ appears in (C4) and is nowhere defined. Two separate questions:

Is it needed mathematically? No. Buffer is scarce — (C3) binds — so the solver has no reason to place buffer where it cannot be used. Allocating more than $\max_{\omega}(L_{i-1}^{\omega} + \delta_i^{\omega})$ to segment i is provably wasted, because beyond that point the segment already absorbs everything reaching it in every scenario, and the budget would be better spent elsewhere.

Is it needed operationally? Yes, if the answer is meant to be implementable. Without it the optimum may schedule a four-minute segment at eleven minutes, which conflicts with other traffic and is not a timetable anyone would adopt. A defensible choice is a proportional cap, $\bar{b}_i = \alpha m_i$, in the spirit of the running-time supplements timetable planners already use.

Report whether the cap binds. If it does not, it is inert and the result is data-driven. If it does, the result is partly driven by a chosen constant and that must be stated, along with the sensitivity to α . Run once uncapped as the reference.

3. Report both weightings; do not choose one

w_i is a modelling decision and boardings are unavailable. Choosing a plausible-looking weight vector would reintroduce precisely the objection that ruled out the delay-management formulation: an objective function containing invented numbers.

Run both defensible extremes and publish both:

Weighting	w_i	Optimises for
Terminus-only	$w_n = 1$, else 0	end-to-end punctuality
Uniform	$w_i = 1$ for all i	passengers alighting anywhere

They bracket the answer. If the optimised allocation beats b^0 under both, the conclusion is robust to the weighting — a stronger claim than either result alone. If it wins under one and loses under the other, that is a finding about where the gain comes from, and it is reported as such rather than resolved by picking the flattering one.

6. What happened when it was run

The formulation is verified, not asserted

`src/buffer_lp.py` solves the program, then replays the resulting buffer vector through the true nonlinear recursion and reports the difference. It is **0.00e+00** on every run. This is the argument of section 3 made executable: if (C1) and (C2) were too loose, the solver would report a cost the real dynamics cannot achieve, and the two numbers would separate. The self-check makes the same assertion on random data under *both* weightings — including terminus-only, the case where the naive argument fails.

The solver also refuses a negative weight rather than silently returning an optimum to a different problem.

The result: mixed, and mostly not distinguishable from noise

Fitting on early days, evaluating on later ones, cost in weighted seconds of lateness:

Train	Route	Segs	Days	Fit/seg	Weighting	Base	Opt	Diff	Sig.
D808	Connolly→Drogheda	11	30	1.36	terminus	12.4	6.0	+6.4	no
D808					uniform	78.4	51.6	+26.8	no
E230	Malahide→Bray	27	28	0.52	uniform	461.1	102.9	+358.3	yes
E921	Bray→Howth	27	24	0.44	uniform	146.0	169.9	−23.9	no
E828	Bray→Malahide	27	23	0.41	uniform	911.1	1211.0	−299.9	no
E930	Bray→Howth	27	22	0.41	uniform	1034.2	1294.4	−260.2	no

The governing quantity is fitting days per decision variable. Above 1, the optimiser improves. At 0.4, it loses. On E828 the program drives the fit-set objective to **exactly zero** — 27 free buffers absorb every delay across 11 days completely — and then costs 300s on days it has not seen. That is a model with more freedom than evidence, which is overfitting in its purest form.

E230 is the one significant result and is not read as a counterexample. Its terminus-weighted baseline is 0.0s — the train is never late at Bray on the test days — so the entire gain sits at intermediate stops on a route whose timetable happens to be badly balanced. One significant result out of twelve, unadjusted for having run twelve, is not a finding.

The sensitivity that settles it

D808 is the only instance improving under both weightings. Moving the split:

Holdout	Fit days	Uniform diff	95% CI
0.3	21	+36.0 s	[−97.3, +10.7]
0.4	18	+26.5 s	[−82.5, +14.0]
0.5	15	+26.8 s	[−80.8, +10.4]
0.6	12	−197.7 s	[+10.3, +516.7]

The sign flips at twelve fitting days. Any claim resting on this instance would be a claim about where the split was drawn, and is reported as such.

Why so little is significant

The bootstrap resamples whole *days*, for the reason in section 5. Twelve to twenty-one resampled days cannot resolve a 27-second difference. The number governing the statistics is the number of

days, not the three hundred-odd lateness variables in the program.

7. The fix: shrinkage toward the timetable

If the diagnosis is overfitting, the standard remedy is to stop the optimiser moving freely and make it *pay* for every second it departs from the existing allocation:

$$\min \quad \frac{1}{|\Omega|} \sum_{\omega} \sum_i w_i L_i^{\omega} + \lambda \sum_i |b_i - b_i^0|$$

The same linearisation, a second time

An absolute value is no more linear than a maximum, and it is handled the same way:

$$(C6) \quad u_i \geq b_i - b_i^0, \quad (C7) \quad u_i \geq b_i^0 - b_i.$$

Two inequalities, no equality. u_i carries cost $\lambda > 0$ in the objective and appears nowhere else, so the solver drives it down onto $\max(b_i - b_i^0, b_i^0 - b_i) = |b_i - b_i^0|$. Nothing rewards a larger u_i , exactly as nothing rewarded a larger L_i^{ω} . **The tightness argument of section 3 transfers verbatim** — worth knowing as a second instance of the technique rather than a trick reused once.

$\lambda = 0$ recovers the unpenalised program; $\lambda \rightarrow \infty$ pins $b = b^0$, which *is* the baseline. The sweep therefore runs continuously between the two things being compared, and the baseline appears as its own limiting case rather than as a separate number.

It works, and the shape of the curve is the evidence

E828, the worst overfitter, uniform weighting:

λ	Held-out	vs baseline	$\sum_i b_i - b_i^0 $	Sig.
0.00	1211.0 s	−299.9 s	1530 s	no
0.01	677.0 s	+234.1 s	1044 s	yes
0.03	677.0 s	+234.1 s	1044 s	yes
0.10	759.0 s	+152.1 s	528 s	no
0.30	711.1 s	+200.0 s	192 s	yes
1.00	846.5 s	+64.5 s	60 s	yes
3.00+	911.1 s	0.0 s	0 s	(pinned to baseline)

The curve is U-shaped, and that shape is the proof of the diagnosis. Worse at $\lambda = 0$ (overfits), worse as λ grows (becomes the baseline), best in between. Had the optimiser simply had nothing to add, held-out cost would fall monotonically toward the baseline and never beat it. It does not: it beats the baseline by 234s in the middle.

And it generalises

Uniform weighting, best λ :

Train	Segs	Days	$\lambda = 0$	Best	Sig.	Baseline
D808	11	30	+26.8 s	+26.8 s ($\lambda = 0$)	no	78.4 s
E230	27	28	+358.3 s	+ 407.6 s	yes	461.1 s
E921	27	24	−23.9 s	+36.5 s	no	146.0 s
E828	27	23	−299.9 s	+ 234.1 s	yes	911.1 s
E930	27	22	−260.2 s	+ 142.4 s	yes	1034.2 s

Every train improves. All three that *lost* unpenalised become positive. Three of five are significant against the day-level bootstrap, which is the conservative test. D808 is the one instance needing no shrinkage — and it is also the only one whose fitting days exceed its decision variables, which is the same diagnosis seen from the other side.

The finding that matters more than the number

Under **terminus-only** weighting, shrinkage never helps: the best λ is always zero, and on three of five trains the baseline terminal lateness is **0.0 s**. These trains arrive at their final stop on time, essentially always. All of the available gain is at *intermediate* stops.

The timetable’s padding is well placed for terminal punctuality and badly placed for everyone getting off in the middle. That is not a criticism of the timetable — terminal punctuality is what operators are measured on, so it is what the padding has been tuned for.

So the claim is not “the timetable is suboptimal”. It is: *the timetable optimises terminal punctuality, and a passenger-weighted objective wants the padding somewhere else*. That is defensible directly from the terminus-versus-uniform split, and it is the reason section 5 insisted on running both rather than picking one.

One honest caveat on these gains

λ is now a hyperparameter, and it was chosen on the same held-out days used to report the improvement. With this few days there is no room for a third split, so **the reported gains are optimistic** by however much selecting on the evaluation set inflates them. The remedy is the same as before and is still outstanding: more days. The archive holds 34 dates because that is what was backfilled; the source serves history to 2007.

The qualitative result, which stands regardless

D808 under uniform weighting, buffer per segment in seconds:

timetable	319	54	48	141	72	108	132	126	102	63	192
optimised	391	54	18	459	42	18	12	30	18	27	288

The optimiser strips the middle of the route almost bare and concentrates padding at the first, fourth and last segments: absorb early, absorb before the terminus. That is a coherent strategy, and it is also exactly the concentration the cap in (C4) exists to restrain — this was run uncapped, so the cap binds nowhere.

The whole arc, which is what to tell. *I formulated it as a linear program and verified the linearisation is exact by replaying the solution through the true nonlinear recursion. Evaluated on held-out days it lost — badly, on the long routes. The diagnosis was overfitting: twenty-seven decision variables against eleven fitting scenarios, and the objective went to exactly zero on the fitting set. The remedy was an L1 penalty on departing from the existing timetable, linearised the same way the max was. That turned a 300-second loss into a 234-second sig-*

nificant gain, and the held-out cost is U-shaped in the penalty weight, which is what confirms the diagnosis rather than merely fitting it. The gain sits entirely at intermediate stops: the timetable is already tuned for terminal punctuality.

Every step of that is checkable, and the failure came before the fix — which is the part that makes it an engineering story rather than a result.

8. Questions to expect

1. *How is this optimization rather than calling a library?* — Answer with the formulation: variables, objective, constraints, and the linearisation of the max. The solver is arithmetic; the modelling is section 3.
2. *Your constraints are inequalities but the dynamics are an equality. Why is that valid?* — The three-step argument, then the $\sum_{j \geq i} w_j > 0$ condition.
3. *What breaks it?* — A negative weight. Explain the inflation mechanism.
4. *Why is this a linear program and not an integer program?* — Every variable is continuous; there is no discrete decision. Delay management would be an integer program, because hold-or-drop is binary — and it was rejected for lack of data, not for difficulty.
5. *How do you know the improvement is real?* — Paired bootstrap over days, and the sample size is days.
6. *What are its assumptions?* — Primary delays exogenous (the largest threat: drivers pace to the timetable, so added buffer may attract a slower run); minimum running time estimated from a sample quantile; single train, no network capacity; counterfactual, so simulated rather than validated.
7. *Does it assume segments are independent?* — No. Scenarios are whole days, so within-day correlation is preserved exactly. This is a genuine advantage of sample average approximation over a parametric model.
8. *Did it work?* — Not at first, and say that first. The long routes overfit and lost. Then the diagnosis (fitting days per decision variable), then the remedy (an L1 penalty on departing from the timetable, linearised the same way the max was), then the result: every train improves, three of five significantly.
9. *How do you know the shrinkage fixed overfitting rather than just fitting better?* — The held-out cost is U-shaped in λ . If the optimiser had nothing to add it would fall monotonically to the baseline and never beat it.
10. *Where does the gain come from?* — Entirely from intermediate stops. Under terminus-only weighting there is nothing to win, because these trains already arrive on time at their terminus. The timetable is tuned for the metric operators are measured on.

If the optimiser does not beat the existing timetable, that result is recorded and kept. A negative result honestly measured is worth more than a claim that has to be hedged.